

WEEK 11 PRACTICAL

QUESTION-1: Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

AIM

To create a multithreaded counter using POSIX threads and demonstrate a race condition while using thread-local storage.

Program

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define NUM_THREADS 4
#define ITERATIONS 10000

volatile int counter = 0;
/* Thread-local variable */
_Thread_local int thread_local_value;
void *increment_counter(void *arg)
{
    int id = *(int *)arg;
    thread_local_value = id;

    for (int i = 0; i < ITERATIONS; i++) {
        int temp = counter;
        usleep(1);           // makes race condition easier to observe
        counter = temp + 1;
    }

    printf("Thread %d: Thread-local value = %d\n",
           id, thread_local_value);
    pthread_exit(NULL);
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS];

    printf("Starting %d threads...\n\n", NUM_THREADS);

    for (int i = 0; i < NUM_THREADS; i++) {
        thread_ids[i] = i + 1;
        pthread_create(&threads[i], NULL,
                      increment_counter, &thread_ids[i]);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\nExpected counter value : %d\n",
           NUM_THREADS * ITERATIONS);
    printf("Actual counter value   : %d\n", counter);

    return 0;
}

```

OUTPUT

```
ashborn@DESKTOP-I4G5Q4N:~/week11_p$ ./q1
Starting 4 threads...

Thread 2: Thread-local value = 2
Thread 4: Thread-local value = 4
Thread 3: Thread-local value = 3
Thread 1: Thread-local value = 1

Expected counter value : 40000
Actual counter value   : 10575
```

OBSERVATION

Multiple threads accessed and updated the same shared counter without synchronization. Because the threads can read and modify the variable concurrently, some updates are lost, resulting in an actual value different from the expected value. The thread-local variable was maintained separately for each thread.

RESULT

A POSIX multithreaded counter was successfully implemented using `pthread_create()`, `pthread_join()`, and `pthread_exit()`, and the race condition was demonstrated successfully.

QUESTION-2: Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

AIM

To eliminate the race condition in the multithreaded counter using a POSIX mutex and compare the synchronized and unsynchronized results.

PROGRAM

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 4
#define ITERATIONS 10000

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
void *increment_counter(void *arg)
{
    int id = *(int *)arg;
    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }

    printf("Thread %d completed\n", id);
    pthread_exit(NULL);
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int thread_ids[NUM_THREADS];

    printf("Starting %d threads with mutex...\n\n", NUM_THREADS);

    for (int i = 0; i < NUM_THREADS; i++) {
        thread_ids[i] = i + 1;

        pthread_create(&threads[i],
                      NULL,
                      increment_counter,
                      &thread_ids[i]);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("\nExpected counter value : %d\n",
          NUM_THREADS * ITERATIONS);
    printf("Actual counter value : %d\n", counter);
    pthread_mutex_destroy(&lock);

    return 0;
}

```

OUTPUT

```
ashborn@DESKTOP-I4G5Q4N:~/week11_p$ ./q2
Starting 4 threads with mutex...

Thread 1 completed
Thread 4 completed
Thread 2 completed
Thread 3 completed

Expected counter value : 40000
Actual counter value   : 40000
```

OBSERVATION

The mutex ensures that only one thread can update the shared counter at a time. Therefore, the critical section is protected and lost updates are prevented. Compared with the race-condition version, synchronization produces a consistent and correct counter value.

RESULT

The counter program was successfully synchronized using `pthread_mutex_lock()` and `pthread_mutex_unlock()`. The race condition was eliminated and the expected counter value was obtained.